

DRL Assignment details: PART #2 - DP .

Autonomous Drone Rescue Using Dynamic Programming.

Team # 116

Student Name	Student ID
PRASAD SHIVAJI KULKARNI	2025aa05444@wilp.bits-pilani.ac.in
SHELAR SACHIN KRISHNA	2025aa05387@wilp.bits-pilani.ac.in
POWAR SAGAR GANPATI	2025aa05421@wilp.bits-pilani.ac.in
SUJEET KUMAR YADAV	2025aa05326@wilp.bits-pilani.ac.in
JAMDAR SARTHAK VITHALDAS	2025aa05112@wilp.bits-pilani.ac.in

Student Collab Image:

L-R: Sagar, Sarthak, Sujeet.

Prasad, Sachin.

**Student Contributions Sheet:**

Student Name	Student Contribution
PRASAD SHIVAJI KULKARNI	Developed drone movement and battery consumption logic. Assisted in testing environment transitions and validating reward calculations.
SHELAR SACHIN KRISHNA	Implemented charging station, rescue target, and obstacle handling features. Performed debugging and execution validation for multiple scenarios.
POWAR SAGAR GANPATI	Worked on Value Iteration implementation and policy computation. Analyzed convergence behavior and optimized state space processing.
SUJEET KUMAR YADAV	Managed virtual machine execution, captured timestamped screenshots, and prepared final documentation. Consolidated outputs and verified assignment requirements before submission.
JAMDAR SARTHAK VITHALDAS	Created policy visualization, heatmaps, and graphical representation of results. Contributed to performance analysis and interpretation of observations.

Python Code for Part#2 DP.

BITS WILP - Deep Reinforcement Learning | Lab Assignment 1 - Part 2 (DP)

Autonomous Drone Rescue Using Dynamic Programming (Value Iteration)

ID : 2025aa05112 # Taking as power the given Assignment Instructions.

ID-derived configuration (last digit = 2):

- Grid size : 5 x 5
- Rescue targets (R) : 2
- Charging stations : 1
- Danger zones (D) : 3
- Blocked cells (X) : 2
- Wind zones (W) : 2 (placed deterministically from student ID)
- Max battery : 10 (even last digit)
- Wind probability : 20% (last digit 0-4)
- Max episode steps : 50

Expected outcomes covered:

1. Custom Drone Rescue Environment (reset, step, render)
2. Dynamic Programming — Value Iteration (theta = 1e-3)
3. Policy Visualisation
4. State-Value Heatmap Analysis
5. DP Scalability Discussion

=====Python Code Start=====

```
import os
import platform
import socket
import time
from collections import defaultdict
from datetime import datetime

import matplotlib.pyplot as plt
import numpy as np

# =====
# Virtual Lab Metadata (printed at execution start per assignment requirement)
# =====

EXECUTION_TIMESTAMP = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
HOSTNAME = socket.gethostname()
VM_ID = (
    os.environ.get("VM_ID") # To Fetch the HostName
    or os.environ.get("COMPUTERNAME")
    or platform.node()
    or HOSTNAME
)

print("=" * 70)
print("VIRTUAL LAB EXECUTION METADATA")
print("=" * 70)
```

```

print(f"Execution Timestamp : {EXECUTION_TIMESTAMP}")
print(f"Virtual Machine ID : {VM_ID}")
print(f"Hostname : {HOSTNAME}")
print(f"Platform : {platform.platform()}")
print("=" * 70)

```

Student ID and environment parameters

```
# =====
```

List of students and their IDs

```

# Prasad - 2025aa05444@wilp.bits-pilani.ac.in
# Sagar - 2025aa05421@wilp.bits-pilani.ac.in
# Sachin - 2025aa05387@wilp.bits-pilani.ac.in
# Sujeet - 2025aa05326@wilp.bits-pilani.ac.in
# Sarthak - 2025aa05112@wilp.bits-pilani.ac.in

```

Based on sorting 2025aa05112 selected as the student ID

```

STUDENT_ID = "2025aa05112"
LAST_DIGIT = int(STUDENT_ID[-1]) # 2

```

Grid and object counts (last digit 0-4)

```

GRID_SIZE = 5 if LAST_DIGIT <= 4 else 6
NUM_RESCUE = 2 if LAST_DIGIT <= 4 else 3
NUM_CHARGE = 1 if LAST_DIGIT <= 4 else 2
NUM_DANGER = 3 if LAST_DIGIT <= 4 else 4
NUM_BLOCKED = 2 if LAST_DIGIT <= 4 else 3
NUM_WIND = 2 # wind cells added from ID-based placement (documented below)

```

```

MAX_BATTERY = 10 if (LAST_DIGIT % 2 == 0) else 15
WIND_PROB = 0.20 if LAST_DIGIT <= 4 else 0.30
MAX_STEPS = 50 if GRID_SIZE == 5 else 75

```

Reward table (assignment specification)

```

REWARD_RESCUE = 20
REWARD_DANGER = -10
REWARD_BATTERY_DEAD = -20
REWARD_CHARGE = 5
REWARD_MOVE = -1

```

Value Iteration stopping threshold

```

THETA = 1e-3 # stopping threshold for value iteration
GAMMA = 0.99 # discount factor (standard episodic MDP; documented in output)

```

Action indices and names

```

ACTION_UP, ACTION_DOWN, ACTION_LEFT, ACTION_RIGHT, ACTION_HOVER = 0, 1, 2, 3, 4
ACTION_NAMES = ["Up", "Down", "Left", "Right", "Hover"]
ACTION_DELTAS = {
    ACTION_UP: (-1, 0),
    ACTION_DOWN: (1, 0),
    ACTION_LEFT: (0, -1),
    ACTION_RIGHT: (0, 1),
    ACTION_HOVER: (0, 0),
}
MOVEMENT_ACTIONS = [ACTION_UP, ACTION_DOWN, ACTION_LEFT, ACTION_RIGHT]

```

Cell type constants

```

CELL_START = "S"
CELL_FREE = "F"
CELL_DANGER = "D"
CELL_RESCUE = "R"
CELL_CHARGE = "C"
CELL_WIND = "W"
CELL_BLOCKED = "X"

```

```
def _student_numeric_seed(student_id: str) -> int:
```

```
    """Derive a reproducible integer seed from all digits in the student ID."""
```

```
    digits = "".join(ch for ch in student_id if ch.isdigit())
    return int(digits) if digits else 0

```

```
def build_grid_layout(student_id: str, grid_size: int) -> dict:
```

```
    """
```

```
    Place environment symbols on the grid using a deterministic function of
    student ID. Start (S) is fixed at top-left (0, 0).
```

```
    Placement order: blocked -> danger -> wind -> charging -> rescue.
```

```
    Remaining cells are free (F).
```

```
    At least one cell among (0, 1) and (1, 0) must stay passable so the drone
    can leave the start corner (assignment: S is top-left, drone must move).
```

```
    """
```

```
    seed = _student_numeric_seed(student_id)
    occupied = {(0, 0)}
    layout = {}

```

Cells that cannot be blocked — preserve an exit corridor from S

```
no_block_cells = {(0, 1), (1, 0)}
```

Candidate cells sorted by ID hash (excluding start)

```

candidates = [
    (r, c)
    for r in range(grid_size)
    for c in range(grid_size)
    if (r, c) != (0, 0)
]
candidates.sort(key=lambda rc: (rc[0] * 31 + rc[1] * 17 + seed) % 997)

```

```
def place(count, symbol, forbidden=None):
```

```
    forbidden = forbidden or set()
```

```
    placed = []
```

```
    for cell in candidates:
```

```
        if len(placed) >= count:
```

```
            break
```

```
        if cell in occupied or cell in forbidden:
```

```
            continue
```

```
        occupied.add(cell)
```

```
        layout[cell] = symbol

```

```

    placed.append(cell)
    return placed

```

```

blocked = place(NUM_BLOCKED, CELL_BLOCKED, forbidden=no_block_cells)
dangers = place(NUM_DANGER, CELL_DANGER)
winds = place(NUM_WIND, CELL_WIND)
charges = place(NUM_CHARGE, CELL_CHARGE)
rescues = place(NUM_RESCUE, CELL_RESCUE)

```

```

return {
    "blocked": blocked,
    "dangers": dangers,
    "winds": winds,
    "charges": charges,
    "rescues": rescues,
    "layout": layout,
}

```

```

GRID_INFO = build_grid_layout(STUDENT_ID, GRID_SIZE)
RESCUE_POSITIONS = tuple(sorted(GRID_INFO["rescues"]))
CHARGE_POSITIONS = tuple(sorted(GRID_INFO["charges"]))
NUM_RESCUE = len(RESCUE_POSITIONS)

```

```

# =====
# Step 1: Custom Drone Rescue Environment
# =====
#Creating the class for DroneRescue

```

```

class DroneRescueEnv:

```

```

    """

```

```

    Grid-world MDP for autonomous drone rescue.

```

```

    State (internal):

```

```

        position      : (row, col)
        battery       : int in [0, MAX_BATTERY]
        rescued_mask   : bitmask — bit i set if rescue target i is saved

```

```

    The environment implements reset(), step(action), render(), and
    get_valid_actions() for interactive rollouts after DP.

```

```

    """

```

```

def __init__(self):
    self.grid_size = GRID_SIZE
    self.max_battery = MAX_BATTERY
    self.wind_prob = WIND_PROB
    self.max_steps = MAX_STEPS
    self.rescue_positions = list(RESCUE_POSITIONS)
    self.charge_positions = set(CHARGE_POSITIONS)
    self.blocked_positions = set(GRID_INFO["blocked"])
    self.danger_positions = set(GRID_INFO["dangers"])
    self.wind_positions = set(GRID_INFO["winds"])
    self.static_layout = dict(GRID_INFO["layout"])

```

```

self.rng = np.random.default_rng(_student_numeric_seed(STUDENT_ID))
self.reset()

def _cell_symbol(self, pos):
    """Return static map symbol at position (before dynamic rescue removal)."""
    if pos == (0, 0):
        return CELL_START
    if pos in self.blocked_positions:
        return CELL_BLOCKED
    if pos in self.rescued_positions:
        return CELL_FREE
    return self.static_layout.get(pos, CELL_FREE)

def reset(self):
    """Reset episode: drone at S, full battery, all targets active."""
    self.position = (0, 0)
    self.battery = self.max_battery
    self.rescued_mask = 0
    self.rescued_positions = set()
    self.steps = 0
    self.total_reward = 0.0
    self.done = False
    self.history = [self.position]
    return self._get_observation()

def _get_observation(self):
    """Observation dict for logging and analysis."""
    return {
        "position": self.position,
        "battery": self.battery,
        "rescued_mask": self.rescued_mask,
        "rescued_positions": set(self.rescued_positions),
        "steps": self.steps,
        "done": self.done,
    }

def get_valid_actions(self, position=None, battery=None, done=None):
    """
    Return action indices allowed at the current (or supplied) state.
    All five actions are always valid; blocked-cell collisions are handled
    in the transition (drone stays in place, battery still consumed).
    """
    if done is None:
        done = self.done
    if done:
        return []
    if battery is not None and battery <= 0:
        return []
    return list(range(5))

def _apply_movement(self, action):
    """Compute intended next position for a movement action."""
    dr, dc = ACTION_DELTAS[action]
    r, c = self.position

```

```

nr, nc = r + dr, c + dc
if not (0 <= nr < self.grid_size and 0 <= nc < self.grid_size):
    return self.position
if (nr, nc) in self.blocked_positions:
    return self.position
return (nr, nc)

def _resolve_action(self, action):
    """
    Wind rule: on cell W, movement actions may be redirected uniformly
    to one of {Up, Down, Left, Right} with probability WIND_PROB.
    """
    if action == ACTION_HOVER:
        return ACTION_HOVER
    r, c = self.position
    if (r, c) not in self.wind_positions:
        return action
    if self.rng.random() < self.wind_prob:
        return int(self.rng.choice(MOVEMENT_ACTIONS))
    return action

def step(self, action):
    """
    Execute one action. Returns (observation, reward, done, info).

    Order of updates:
    1. Resolve wind (if applicable)
    2. Move / hover
    3. Consume 1 battery (all actions)
    4. Hover on charger: +2 battery (capped)
    5. Enter charger: refill to max (+ reward)
    6. Rescue / danger rewards
    7. Check termination (battery 0, all rescued, max steps)
    """
    if self.done:
        raise RuntimeError("Episode already finished. Call reset() first.")

    reward = REWARD_MOVE
    resolved = self._resolve_action(action)

    if resolved == ACTION_HOVER:
        next_pos = self.position
    else:
        next_pos = self._apply_movement(resolved)

    self.position = next_pos
    self.steps += 1

    # Battery: every action costs 1
    self.battery -= 1

    # Hover on charging station: regain 2 units (net +1 after -1 above)
    if action == ACTION_HOVER and self.position in self.charge_positions:
        self.battery = min(self.max_battery, self.battery + 2)

```

Enter charging station (movement onto C)

```

if self.position in self.charge_positions:
    self.battery = self.max_battery
    reward += REWARD_CHARGE

```

Rescue target

```

for idx, rpos in enumerate(self.rescue_positions):
    if self.position == rpos and not (self.rescued_mask & (1 << idx)):
        reward += REWARD_RESCUE
        self.rescued_mask |= 1 << idx
        self.rescued_positions.add(rpos)

```

Danger zone

```

if self.position in self.danger_positions:
    reward += REWARD_DANGER

```

```

terminated = False
if self.battery <= 0:
    reward += REWARD_BATTERY_DEAD
    terminated = True
elif self.rescued_mask == (1 << NUM_RESCUE) - 1:
    terminated = True
elif self.steps >= self.max_steps:
    terminated = True

```

```

self.total_reward += reward
self.done = terminated
self.history.append(self.position)

```

```

info = {
    "resolved_action": ACTION_NAMES[resolved],
    "wind_applied": resolved != action,
}
return self._get_observation(), reward, terminated, info

```

```

def render(self):

```

```

    """Print ASCII grid with drone position marked as 'A' (agent)."""

```

```

    symbols = {
        CELL_START: "S",
        CELL_FREE: "F",
        CELL_DANGER: "D",
        CELL_RESCUE: "R",
        CELL_CHARGE: "C",
        CELL_WIND: "W",
        CELL_BLOCKED: "X",
    }
    print("\n--- Drone Rescue Grid ---")
    print(
        f"Position={self.position} Battery={self.battery} "
        f"Rescued_mask={self.rescued_mask} Step={self.steps}"
    )
    for r in range(self.grid_size):
        row_chars = []

```



```

    for c in range(self.grid_size):
        if (r, c) == self.position:
            row_chars.append("A")
        else:
            sym = self._cell_symbol((r, c))
            row_chars.append(symbols.get(sym, sym))
    print(" ".join(row_chars))
print("Legend: S=Start F=Free D=Danger R=Rescue C=Charge W=Wind X=Blocked A=Drone")
print("---\n")

```

```

# =====
# Step 2: MDP model for Dynamic Programming (Value Iteration)
# =====

```

```

class DroneRescueMDP:
    """

```

Explicit MDP for Value Iteration.

State representation (hashable tuple):

(row, col, battery, rescued_mask, step)

step counts actions taken so far (0 at start). Episode cap MAX_STEPS is modelled as an absorbing time limit per assignment rules.

Terminal states:

- battery == 0 (after transition)
- all targets rescued (rescued_mask == 2^{NUM_RESCUE} - 1)
- step >= MAX_STEPS

Transition dynamics match DroneRescueEnv rules; wind is modelled as stochastic mixture over movement directions on wind cells.

```

    """

```

```

def __init__(self):
    self.grid_size = GRID_SIZE
    self.max_battery = MAX_BATTERY
    self.wind_prob = WIND_PROB
    self.rescue_positions = list(RESCUE_POSITIONS)
    self.charge_positions = set(CHARGE_POSITIONS)
    self.blocked_positions = set(GRID_INFO["blocked"])
    self.danger_positions = set(GRID_INFO["dangers"])
    self.wind_positions = set(GRID_INFO["winds"])
    self.all_rescued_mask = (1 << NUM_RESCUE) - 1
    self.states = []
    self.terminal_states = set()
    self._build_state_space()

```

```

def _is_valid_position(self, pos):
    r, c = pos
    if not (0 <= r < self.grid_size and 0 <= c < self.grid_size):
        return False
    return pos not in self.blocked_positions

```

```

def _build_state_space(self):
    """Enumerate all (position, battery, rescued_mask, step) tuples."""
    valid_positions = [
        (r, c)
        for r in range(self.grid_size)
        for c in range(self.grid_size)
        if self._is_valid_position((r, c))
    ]
    state_set = set()
    for pos in valid_positions:
        for battery in range(0, self.max_battery + 1):
            for mask in range(1 << NUM_RESCUE):
                for step in range(0, MAX_STEPS + 1):
                    state = (pos[0], pos[1], battery, mask, step)
                    state_set.add(state)
                    if (
                        battery == 0
                        or mask == self.all_rescued_mask
                        or step >= MAX_STEPS
                    ):
                        self.terminal_states.add(state)
    self.states = sorted(state_set)

def is_terminal(self, state):
    _, _, battery, mask, step = state
    return (
        battery == 0
        or mask == self.all_rescued_mask
        or step >= MAX_STEPS
    )

def get_valid_actions(self, state):
    """Actions allowed in MDP (same as environment)."""
    if self.is_terminal(state):
        return []
    _, _, battery, _, step = state
    if battery <= 0 or step >= MAX_STEPS:
        return []
    return list(range(5))

def _next_position(self, pos, action):
    if action == ACTION_HOVER:
        return pos
    dr, dc = ACTION_DELTAS[action]
    r, c = pos
    npos = (r + dr, c + dc)
    if not self._is_valid_position(npos):
        return pos
    if npos in self.blocked_positions:
        return pos
    return npos

def _transition_outcomes(self, state, action):
    """

```

Return list of (probability, next_state, reward, done) tuples.
Implements wind stochasticity as a uniform mixture over directions.

```

"""
pos = (state[0], state[1])
battery = state[2]
mask = state[3]
step = state[4]

if self.is_terminal(state) or battery <= 0:
    return []

# Build distribution over resolved movement actions
action_probs = defaultdict(float)
if action == ACTION_HOVER:
    action_probs[ACTION_HOVER] = 1.0
elif pos in self.wind_positions:
    p_wind = self.wind_prob
    for mv in MOVEMENT_ACTIONS:
        action_probs[mv] += p_wind / len(MOVEMENT_ACTIONS)
    action_probs[action] += 1.0 - p_wind
else:
    action_probs[action] = 1.0

outcomes = []
for resolved_action, prob in action_probs.items():
    if prob <= 0:
        continue
    next_pos = self._next_position(pos, resolved_action)
    reward = REWARD_MOVE
    next_battery = battery - 1

    if resolved_action == ACTION_HOVER and next_pos in self.charge_positions:
        next_battery = min(self.max_battery, next_battery + 2)

    if next_pos in self.charge_positions:
        next_battery = self.max_battery
        reward += REWARD_CHARGE

    next_mask = mask
    for idx, rpos in enumerate(self.rescue_positions):
        if next_pos == rpos and not (mask & (1 << idx)):
            reward += REWARD_RESCUE
            next_mask |= 1 << idx

    if next_pos in self.danger_positions:
        reward += REWARD_DANGER

    done = False
    if next_battery <= 0:
        reward += REWARD_BATTERY_DEAD
        done = True
        next_battery = 0

    next_step = step + 1

```

```

    if next_mask == self.all_rescued_mask:
        done = True
    if next_step >= MAX_STEPS:
        done = True

    next_state = (
        next_pos[0],
        next_pos[1],
        next_battery,
        next_mask,
        next_step,
    )
    outcomes.append((prob, next_state, reward, done))

return outcomes

def value_iteration(self, gamma=GAMMA, theta=THETA):
    """
    Value Iteration until max_s |V(s) - V_old(s)| < theta.

    Returns V, policy, iterations, final_delta, elapsed_seconds.
    """
    V = {s: 0.0 for s in self.states}
    policy = {s: ACTION_HOVER for s in self.states}

    delta = float("inf")
    iteration = 0
    start = time.perf_counter()

    while delta >= theta:
        delta = 0.0
        iteration += 1
        V_new = dict(V)
        for state in self.states:
            if self.is_terminal(state):
                V_new[state] = 0.0
                continue

            best_q = -float("inf")
            best_action = ACTION_HOVER

            for action in self.get_valid_actions(state):
                q = 0.0
                for prob, next_s, reward, done in self._transition_outcomes(
                    state, action
                ):
                    if done:
                        q += prob * reward
                    else:
                        q += prob * (reward + gamma * V[next_s])
                if q > best_q:
                    best_q = q
                    best_action = action

```

```

V_new[state] = best_q
policy[state] = best_action
delta = max(delta, abs(V_new[state] - V[state]))

```

```

V = V_new

```

```

elapsed = time.perf_counter() - start
return V, policy, iteration, delta, elapsed

```

```

# =====
# Step 3: Policy visualisation helpers
# =====

```

```

ARROW_MAP = {
    ACTION_UP: "\u2191",
    ACTION_DOWN: "\u2193",
    ACTION_LEFT: "\u2190",
    ACTION_RIGHT: "\u2192",
    ACTION_HOVER: "H",
}

```

```

# Colours and labels for diagram legends (assignment cell symbols)

```

```

CELL_STYLE = {
    CELL_START: {"color": "#2e7d32", "label": "S — Start (top-left)"},
    CELL_FREE: {"color": "#e8f5e9", "label": "F — Free / safe"},
    CELL_DANGER: {"color": "#ef9a9a", "label": "D — Danger (-10 reward)"},
    CELL_RESCUE: {"color": "#64b5f6", "label": "R — Rescue target (+20)"},
    CELL_CHARGE: {"color": "#ffd54f", "label": "C — Charging station (+5, refill)"},
    CELL_WIND: {"color": "#4fc3f7", "label": "W — Wind (20% random move)"},
    CELL_BLOCKED: {"color": "#424242", "label": "X — Blocked / obstacle"},
}

```

```

ACTION_STYLE = {
    ACTION_UP: {"symbol": "\u2191", "label": "Up"},
    ACTION_DOWN: {"symbol": "\u2193", "label": "Down"},
    ACTION_LEFT: {"symbol": "\u2190", "label": "Left"},
    ACTION_RIGHT: {"symbol": "\u2192", "label": "Right"},
    ACTION_HOVER: {"symbol": "H", "label": "Hover (recharge on C)"},
}

```

```

# Arrow displacement in display coordinates (length relative to cell)

```

```

_ACTION_OFFSET = {
    ACTION_UP: (0.0, 0.32),
    ACTION_DOWN: (0.0, -0.32),
    ACTION_LEFT: (-0.32, 0.0),
    ACTION_RIGHT: (0.32, 0.0),
    ACTION_HOVER: (0.0, 0.0),
}

```

```

def _cell_type_at(pos, rescued_mask):
    """Return static cell symbol at grid position for plotting."""
    if pos in GRID_INFO["blocked"]:

```

```

    return CELL_BLOCKED
if pos == (0, 0):
    return CELL_START
if pos in RESCUE_POSITIONS:
    idx = list(RESCUE_POSITIONS).index(pos)
    if not (rescued_mask & (1 << idx)):
        return CELL_RESCUE
return GRID_INFO["layout"].get(pos, CELL_FREE)

def _display_xy(row, col):
    """Map grid (row, col) to matplotlib axes coords (x=col, y inverted)."""
    return col + 0.5, GRID_SIZE - 1 - row + 0.5

def _add_cell_type_legend(fig, anchor=(1.02, 0.88)):
    """Legend for terrain / cell types."""
    symbols = [
        CELL_START,
        CELL_FREE,
        CELL_DANGER,
        CELL_RESCUE,
        CELL_CHARGE,
        CELL_WIND,
        CELL_BLOCKED,
    ]
    handles = [
        plt.Rectangle((0, 0), 1, 1, facecolor=CELL_STYLE[sym]["color"], edgecolor="black")
        for sym in symbols
    ]
    labels = [CELL_STYLE[sym]["label"] for sym in symbols]
    fig.legend(
        handles,
        labels,
        loc="upper left",
        bbox_to_anchor=anchor,
        bbox_transform=fig.transFigure,
        fontsize=9,
        title="Cell types",
        title_fontsize=10,
        framealpha=0.95,
    )

def _add_action_legend(fig, anchor=(1.02, 0.38)):
    """Legend for optimal policy action symbols."""
    actions = [ACTION_UP, ACTION_DOWN, ACTION_LEFT, ACTION_RIGHT, ACTION_HOVER]
    handles = [plt.Line2D([0], [0], color="none") for _ in actions]
    labels = [{"ACTION_STYLE[a]['symbol']} {ACTION_STYLE[a]['label']} for a in actions]
    fig.legend(
        handles,
        labels,
        loc="upper left",
        bbox_to_anchor=anchor,
    )

```

```

        bbox_transform=fig.transFigure,
        fontsize=9,
        title="Policy actions ( $\pi^*$ )",
        title_fontsize=10,
        framealpha=0.95,
    )

def _draw_grid_cells(ax, rescued_mask):
    """Draw coloured grid cells with type letter in each cell."""
    from matplotlib.patches import Rectangle

    for r in range(GRID_SIZE):
        for c in range(GRID_SIZE):
            pos = (r, c)
            sym = _cell_type_at(pos, rescued_mask)
            x, y = c, GRID_SIZE - 1 - r
            face = CELL_STYLE[sym]["color"]
            text_color = "white" if sym in (CELL_BLOCKED, CELL_START) else "black"
            ax.add_patch(
                Rectangle(
                    (x, y),
                    1,
                    1,
                    facecolor=face,
                    edgecolor="#212121",
                    linewidth=1.5,
                )
            )
            ax.text(
                x + 0.12,
                y + 0.78,
                sym,
                ha="left",
                va="top",
                fontsize=11,
                fontweight="bold",
                color=text_color,
            )
            ax.text(
                x + 0.5,
                y + 0.08,
                f"({r},{c})",
                ha="center",
                va="bottom",
                fontsize=7,
                color=text_color,
                alpha=0.85,
            )

def print_environment_configuration():
    """Explain grid layout, symbols, and ID-derived parameters."""
    print("\n" + "=" * 70)

```

```

print("ENVIRONMENT CONFIGURATION (Student ID:", STUDENT_ID, ")")
print("=" * 70)
print(f"Last digit of ID          : {LAST_DIGIT}")
print(f"Grid size                  : {GRID_SIZE} x {GRID_SIZE}")
print(f"Rescue targets (R)         : {NUM_RESCUE} at {RESCUE_POSITIONS}")
print(f"Charging stations (C)      : {NUM_CHARGE} at {CHARGE_POSITIONS}")
print(f"Danger zones (D)          : {NUM_DANGER} at {GRID_INFO['dangers']}")
print(f"Blocked cells (X)          : {NUM_BLOCKED} at {GRID_INFO['blocked']}")
print(f"Wind zones (W)             : {NUM_WIND} at {GRID_INFO['winds']}")
print(f"Start position (S)         : (0, 0) top-left (fixed)")
print(f"Initial / max battery       : {MAX_BATTERY}")
print(f"Wind disturbance probability : {WIND_PROB * 100:.0f}% on movement from W")
print(f"Max steps per episode       : {MAX_STEPS}")
print(f"Discount factor (gamma)     : {GAMMA}")
print("\nTransition dynamics (wind):")
print(
    " On cell W, each movement action is redirected uniformly to one of "
    "Up/Down/Left/Right with probability",
    WIND_PROB,
    f"; otherwise the intended direction is used ({1 - WIND_PROB:.0f}%).",
)
print("\nState representation:")
print(" (row, col, battery, rescued_mask, step)")
print(" rescued_mask bit i = 1 iff rescue target i has been saved.")
print(f" step in [0, {MAX_STEPS}] enforces the episode step limit in the MDP.")
print("\nStatic grid map (S fixed at top-left):")
env = DroneRescueEnv()
env.position = (-1, -1)
env.render()

def plot_policy_arrows(policy, V, battery_level, rescued_mask, output_path):
    """
    Draw optimal action arrows on the grid for a fixed battery and rescue status.
    Includes cell-type colours, coordinate labels, and separate legends.
    """
    from matplotlib.patches import FancyArrowPatch

    mdp = DroneRescueMDP()
    fig, ax = plt.subplots(figsize=(10, 8))
    fig.subplots_adjust(right=0.72)

    _draw_grid_cells(ax, rescued_mask)

    for r in range(GRID_SIZE):
        for c in range(GRID_SIZE):
            cx, cy = _display_xy(r, c)
            pos = (r, c)
            sym = _cell_type_at(pos, rescued_mask)
            state = (r, c, battery_level, rescued_mask, 0)

            if sym == CELL_BLOCKED:
                ax.text(
                    cx,

```



```

        cy,
        "NO ENTRY",
        ha="center",
        va="center",
        fontsize=8,
        fontweight="bold",
        color="white",
    )
    continue

if state in policy and not mdp.is_terminal(state):
    action = policy[state]
    dx, dy = _ACTION_OFFSET[action]
    if action == ACTION_HOVER:
        ax.text(
            cx,
            cy,
            "H",
            ha="center",
            va="center",
            fontsize=22,
            fontweight="bold",
            color="#1565c0",
            bbox=dict(
                boxstyle="circle,pad=0.25",
                facecolor="white",
                edgecolor="#1565c0",
                linewidth=2,
            ),
        )
    else:
        ax.add_patch(
            FancyArrowPatch(
                (cx - dx, cy - dy),
                (cx + dx, cy + dy),
                arrowstyle="->",
                mutation_scale=18,
                linewidth=2.5,
                color="#0d47a1",
            )
        )
    elif state in V:
        ax.text(
            cx,
            cy,
            "TERM",
            ha="center",
            va="center",
            fontsize=9,
            color="#616161",
            style="italic",
        )

ax.set_xlim(0, GRID_SIZE)

```

```

ax.set_ylim(0, GRID_SIZE)
ax.set_aspect("equal")
ax.set_xticks(np.arange(GRID_SIZE) + 0.5)
ax.set_xticklabels([str(i) for i in range(GRID_SIZE)])
ax.set_yticks(np.arange(GRID_SIZE) + 0.5)
ax.set_yticklabels([str(GRID_SIZE - 1 - i) for i in range(GRID_SIZE)])
ax.set_xlabel("Column index", fontsize=11)
ax.set_ylabel("Row index (0 = top of grid)", fontsize=11)
ax.set_title(
    "Optimal Policy  $\pi^*(s)$  from Value Iteration",
    fontsize=14,
    fontweight="bold",
    pad=12,
)
ax.text(
    0.5,
    1.02,
    (
        f"Student ID: {STUDENT_ID} | Grid {GRID_SIZE}x{GRID_SIZE} | "
        f"battery={battery_level} | rescued_mask={rescued_mask} | step=0"
    ),
    transform=ax.transAxes,
    ha="center",
    fontsize=10,
    color="#424242",
)

_add_cell_type_legend(fig, anchor=(1.02, 0.88))
_add_action_legend(fig, anchor=(1.02, 0.42))

note = (
    "Arrows show greedy action per cell. Empty/blocked cells cannot be entered. "
    f"Wind probability on W cells: {WIND_PROB:.0%}."
)
fig.text(0.36, 0.02, note, ha="center", fontsize=9, color="#555555")

plt.savefig(output_path, dpi=200, bbox_inches="tight", facecolor="white")
print(f"Policy plot saved: {output_path}")
plt.close()

```

```
def plot_value_heatmap(V, battery_level, rescued_mask, output_path):
```

```
    """
```

```
    Heatmap of  $V^*(s)$  over drone positions with fixed battery and rescue mask.
    Overlays cell symbols, marks blocked cells, and adds readable legends.
```

```
    """
```

```
    from matplotlib.patches import Patch, Rectangle
```

```
    mdp = DroneRescueMDP()
```

```
    values = np.full((GRID_SIZE, GRID_SIZE), np.nan)
```

```
    cell_syms = np.empty((GRID_SIZE, GRID_SIZE), dtype=object)
```

```
    for r in range(GRID_SIZE):
```

```
        for c in range(GRID_SIZE):
```

```

    pos = (r, c)
    cell_syms[r, c] = _cell_type_at(pos, rescued_mask)
    state = (r, c, battery_level, rescued_mask, 0)
    if cell_syms[r, c] == CELL_BLOCKED:
        continue
    if state in V and not mdp.is_terminal(state):
        values[r, c] = V[state]

fig, ax = plt.subplots(figsize=(10, 8))
fig.subplots_adjust(right=0.78)

# Mask blocked cells for display
display = np.ma.masked_invalid(values)
cmap = plt.cm.YlOrRd.copy()
cmap.set_bad(color="#9e9e9e")

vmin = np.nanmin(values) if np.any(~np.isnan(values)) else 0
vmax = np.nanmax(values) if np.any(~np.isnan(values)) else 1
im = ax.imshow(
    display,
    cmap=cmap,
    origin="upper",
    vmin=vmin,
    vmax=vmax,
    extent=(-0.5, GRID_SIZE - 0.5, GRID_SIZE - 0.5, -0.5),
    aspect="equal",
)

cbar = fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
cbar.set_label("Optimal state value  $V^*(s)$ ", fontsize=11)
cbar.ax.tick_params(labels=9)

for r in range(GRID_SIZE):
    for c in range(GRID_SIZE):
        sym = cell_syms[r, c]
        if sym == CELL_BLOCKED:
            ax.add_patch(
                Rectangle(
                    (c - 0.5, r - 0.5),
                    1,
                    1,
                    facecolor="#616161",
                    edgecolor="#212121",
                    linewidth=2,
                    hatch="///",
                    zorder=5,
                )
            )
        ax.text(
            c,
            r,
            "X",
            ha="center",
            va="center",

```

```

        fontsize=14,
        fontweight="bold",
        color="white",
        zorder=6,
    )
    continue

val = values[r, c]
if not np.isnan(val):
    # Contrast text against cell colour
    norm_val = (val - vmin) / (vmax - vmin + 1e-9)
    txt_color = "white" if norm_val > 0.55 else "black"
    ax.text(
        c,
        r + 0.12,
        f"{val:.1f}",
        ha="center",
        va="center",
        fontsize=11,
        fontweight="bold",
        color=txt_color,
        zorder=4,
    )

sym_color = "white" if sym in (CELL_START, CELL_DANGER) else "black"
if sym == CELL_START:
    sym_color = "white"
ax.text(
    c - 0.38,
    r - 0.32,
    sym,
    ha="center",
    va="center",
    fontsize=10,
    fontweight="bold",
    color=sym_color if sym != CELL_DANGER else "white",
    zorder=4,
    bbox=dict(boxstyle="round,pad=0.15", facecolor="black", alpha=0.35)
    if sym == CELL_RESCUE
    else None,
)

ax.set_xticks(range(GRID_SIZE))
ax.set_yticks(range(GRID_SIZE))
ax.set_xlabel("Column index", fontsize=11)
ax.set_ylabel("Row index (0 = top of grid)", fontsize=11)
ax.set_title(
    "State-Value Function  $V^*(s)$  (position slice)",
    fontsize=14,
    fontweight="bold",
    pad=12,
)
ax.text(
    0.5,

```

```

1.02,
(
    f"Student ID: {STUDENT_ID} | Fixed battery={battery_level}, "
    f"rescued_mask={rescued_mask}, step=0 | gamma={GAMMA}"
),
transform=ax.transAxes,
ha="center",
fontsize=10,
color="#424242",
)

# Cell-type legend (right side)
symbols = [
    CELL_START,
    CELL_FREE,
    CELL_DANGER,
    CELL_RESCUE,
    CELL_CHARGE,
    CELL_WIND,
    CELL_BLOCKED,
]
handles = [
    Patch(facecolor=CELL_STYLE[s]["color"], edgecolor="black", label=CELL_STYLE[s]["label"])
    for s in symbols
]
leg_cells = ax.legend(
    handles=handles,
    loc="upper left",
    bbox_to_anchor=(1.18, 1.0),
    fontsize=9,
    title="Cell symbols (corner)",
    title_fontsize=10,
    framealpha=0.95,
)
ax.add_artist(leg_cells)

extra_handles = [
    Patch(facecolor="#9e9e9e", hatch="///", edgecolor="black", label="Blocked (no V value)"),
    Patch(facecolor="white", edgecolor="black", label="Bold number = V*(s)"),
]
ax.legend(
    handles=extra_handles,
    loc="lower left",
    bbox_to_anchor=(1.18, 0.0),
    fontsize=9,
    title="Heatmap notes",
    title_fontsize=10,
    framealpha=0.95,
)

fig.text(
    0.5,
    0.01,
    "Higher V* (red) = more expected future reward from that position under optimal policy.",

```

```

    ha="center",
    fontsize=9,
    color="#555555",
)

plt.savefig(output_path, dpi=200, bbox_inches="tight", facecolor="white")
print(f"Value heatmap saved: {output_path}")
plt.close()

```

```

def simulate_optimal_episode(policy):
    """Roll out greedy policy in environment; return trajectory for discussion."""
    env = DroneRescueEnv()
    obs = env.reset()
    trajectory = [("start", obs["position"], obs["battery"], 0.0)]

    while not env.done:
        state = (
            obs["position"][0],
            obs["position"][1],
            obs["battery"],
            obs["rescued_mask"],
            env.steps,
        )
        mdp = DroneRescueMDP()
        if mdp.is_terminal(state):
            break
        action = policy.get(state, ACTION_HOVER)
        obs, reward, done, info = env.step(action)
        trajectory.append(
            (ACTION_NAMES[action], obs["position"], obs["battery"], reward)
        )

    return env, trajectory

```

```

def print_scalability_discussion():
    """Expected outcome 5: curse of dimensionality and Deep RL relation."""
    print("\n" + "=" * 70)
    print("DP SCALABILITY DISCUSSION")
    print("=" * 70)
    n_pos = GRID_SIZE * GRID_SIZE - NUM_BLOCKED
    n_battery = MAX_BATTERY + 1
    n_mask = 1 << NUM_RESCUE
    approx_states = n_pos * n_battery * n_mask * (MAX_STEPS + 1)
    print(f"Current explicit state count (approx.): {approx_states}")

    print(
        """

```

Curse of dimensionality:

Value Iteration stores $V(s)$ and updates every state. If the grid grows to 10x10, positions alone multiply by 4 (relative to 5x5). Adding more rescue targets doubles the rescue bitmask for each extra target. Dynamic weather would add another state variable (or hidden mode), multiplying storage and

backup cost per sweep.

For 10x10 with 5 rescues, 3 chargers, and weather modes, $|S|$ can exceed millions — each iteration touches all states and all actions, so runtime grows roughly $O(|S| \times |A| \times \text{iterations})$.

Is DP sufficient?

For this 5x5 teaching MDP, DP is appropriate and finds an exact optimal policy. For real autonomous drones with continuous pose, partial observability, moving obstacles, and high-dimensional sensors, tabular DP is infeasible.

How Deep RL helps:

Function approximation (DQN, PPO, actor-critic) generalises across similar states without enumerating the full grid. Experience replay and simulation (digital twins) scale learning to large state spaces.

Real-world relation:

Disaster-response drones must trade off rescue value, energy, and risk under uncertainty — similar to our wind model. Production systems combine learned policies with safety shields and online replanning rather than offline VI on the full physical state space.

""""

)

Main execution — all assignment steps

=====

def main():

 # --- Configuration explanation ---
 print_environment_configuration()

 # --- Build MDP and run Value Iteration ---
 print("=" * 70)
 print("VALUE ITERATION (Dynamic Programming)")
 print("=" * 70)
 print("Stopping threshold theta =", THETA)

 mdp = DroneRescueMDP()
 print(f"Enumerated reachable states: {len(mdp.states)}")
 print(f"Terminal states : {len(mdp.terminal_states)}")

 V, policy, iterations, final_delta, elapsed = mdp.value_iteration()

 print(f"\nConvergence iterations : {iterations}")
 print(f"Final delta (Bellman) : {final_delta:.6e}")
 print(f"Runtime (seconds) : {elapsed:.4f}")

 # Sample optimal values at start
 start_state = (0, 0, MAX_BATTERY, 0, 0)
 print(f"\nV*(start) at full battery, no rescues: {V[start_state]:.4f}")
 print(f"pi*(start) action : {ACTION_NAMES[policy[start_state]]}")

 out_dir = os.path.dirname(os.path.abspath(__file__))

```
# --- Policy visualisation (full battery, no rescues) ---
policy_path = os.path.join(out_dir, f"{STUDENT_ID}_policy.png")
plot_policy_arrows(policy, V, battery_level=MAX_BATTERY, rescued_mask=0, output_path=policy_path)

# --- Value heatmap (fix battery=8, no rescues) ---
heatmap_battery = min(8, MAX_BATTERY)
heatmap_path = os.path.join(out_dir, f"{STUDENT_ID}_value_heatmap.png")
plot_value_heatmap(V, battery_level=heatmap_battery, rescued_mask=0, output_path=heatmap_path)

print("\n" + "=" * 70)
print("STATE-VALUE ANALYSIS (patterns)")
print("=" * 70)
print(
    f"""
```

Heatmap and policy slices use fixed rescued_mask=0 and battery={heatmap_battery}.

Observed patterns:

1. High V^* near rescue targets (R) — the +20 rescue reward propagates backward.
2. Lower V^* near danger (D) because -10 penalises entering those cells unless a rescue path forces a detour.
3. Charging station (C) raises value in its neighbourhood (+5 entry, full refill) when battery is mid-level, encouraging recharge before long detours.
4. Wind cells (W) reduce certainty; optimal policy may prefer safer routes when battery is low because stochastic drift wastes energy.
5. As rescued_mask increases, remaining targets dominate policy arrows toward the nearest uncleared R.

```
"""
```

```
)
```

```
# --- Simulate one greedy episode under optimal policy ---
print("=" * 70)
print("OPTIMAL POLICY ROLLOUT (environment simulation)")
print("=" * 70)
env, trajectory = simulate_optimal_episode(policy)
print("Rescue sequence (action -> position, battery, step reward):")
for step in trajectory:
    print(f" {step}")
print(f"Total episode reward: {env.total_reward:.2f}")
print(f"Steps taken      : {env.steps}")
print(f"Rescued mask       : {env.rescued_mask}")
env.render()
```

```
# --- Scalability ---
print_scalability_discussion()
```

```
print("\n" + "=" * 70)
print("ASSIGNMENT PART 2 COMPLETE")
print("=" * 70)
```

```
if __name__ == "__main__":
    main()
```

Screen Shot of Output from Virtual Lab:


```

Applications: Firefox
run_116-DP
localhost:8888/notebooks/SEM2/run_116-DP.ipynb?
140%
jupyter run_116-DP Last Checkpoint: 3 minutes ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] *
[*]: %run 116_DP.py

=====
VIRTUAL LAB EXECUTION METADATA
=====
Execution Timestamp : 2026-06-03 12:34:41
Virtual Machine ID : 2025aa05326
Hostname : 2025aa05326
Platform : Linux-5.14.0-503.14.1.el9_5.x86_64-with-glibc2.34
=====

=====
ENVIRONMENT CONFIGURATION (Student ID: 2025aa05112 )
=====
Last digit of ID : 2
Grid size : 5 x 5
Rescue targets (R) : 2 at ((1, 3), (2, 1))
Charging stations (C) : 1 at ((0, 4),)
Danger zones (D) : 3 at [(0, 1), (1, 0), (0, 3)]
Blocked cells (X) : 2 at [(0, 2), (1, 1)]
Wind zones (W) : 2 at [(2, 0), (1, 2)]
Start position (S) : (0, 0) top-left (fixed)
Initial / max battery : 10
Wind disturbance probability : 20% on movement from W
Max steps per episode : 50
Discount factor (gamma) : 0.99

Transition dynamics (wind):
  On cell W, each movement action is redirected uniformly to one of Up/Down/Left/Right with probability 0.2 ; otherwise the intended direction is used (80%).

State representation:
  (row, col, battery, rescued_mask, step)
  rescued_mask bit i = 1 iff rescue target i has been saved.
  step in [0, 50] enforces the episode step limit in the MDP.

Static grid map (S fixed at top-left):

```

Static grid map (S fixed at top-left):

--- Drone Rescue Grid ---

Position=(-1, -1) Battery=10 Rescued_mask=0 Step=0

S D X D C

D X W R F

W R F F F

F F F F F

F F F F F

Legend: S=Start F=Free D=Danger R=Rescue C=Charge W=Wind X=Blocked A=Drone

VALUE ITERATION (Dynamic Programming)

Stopping threshold theta = 0.001

Enumerated reachable states: 51612

Terminal states : 17112

Convergence iterations : 51

Final delta (Bellman) : 0.000000e+00

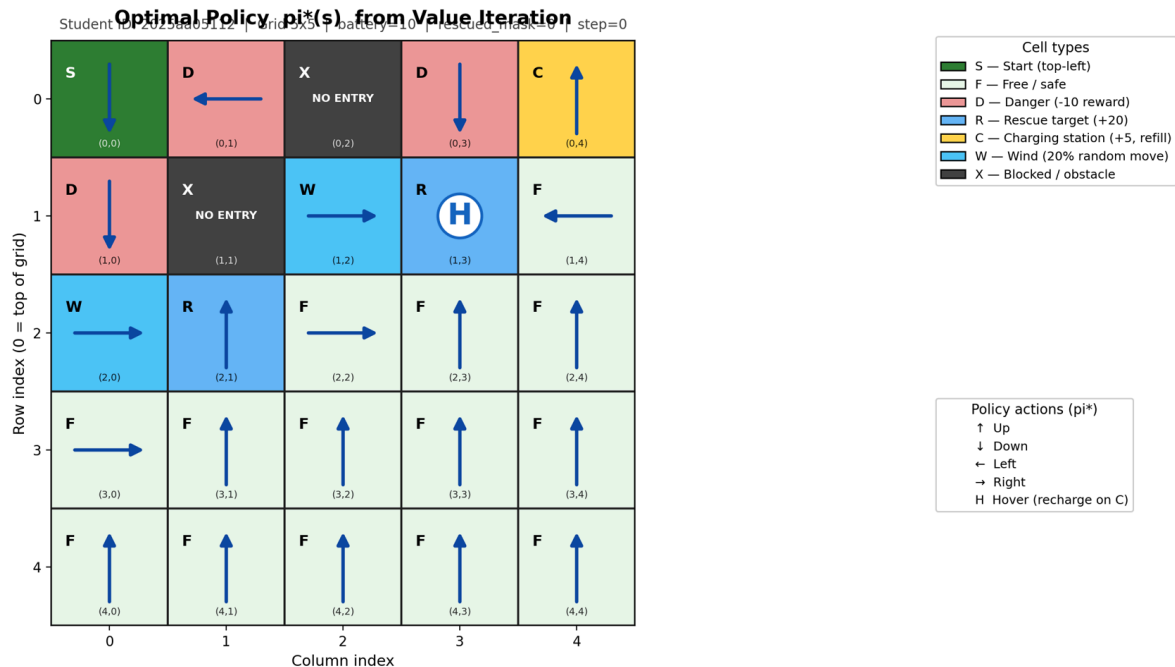
Runtime (seconds) : 43.6951

V*(start) at full battery, no rescues: 136.3561

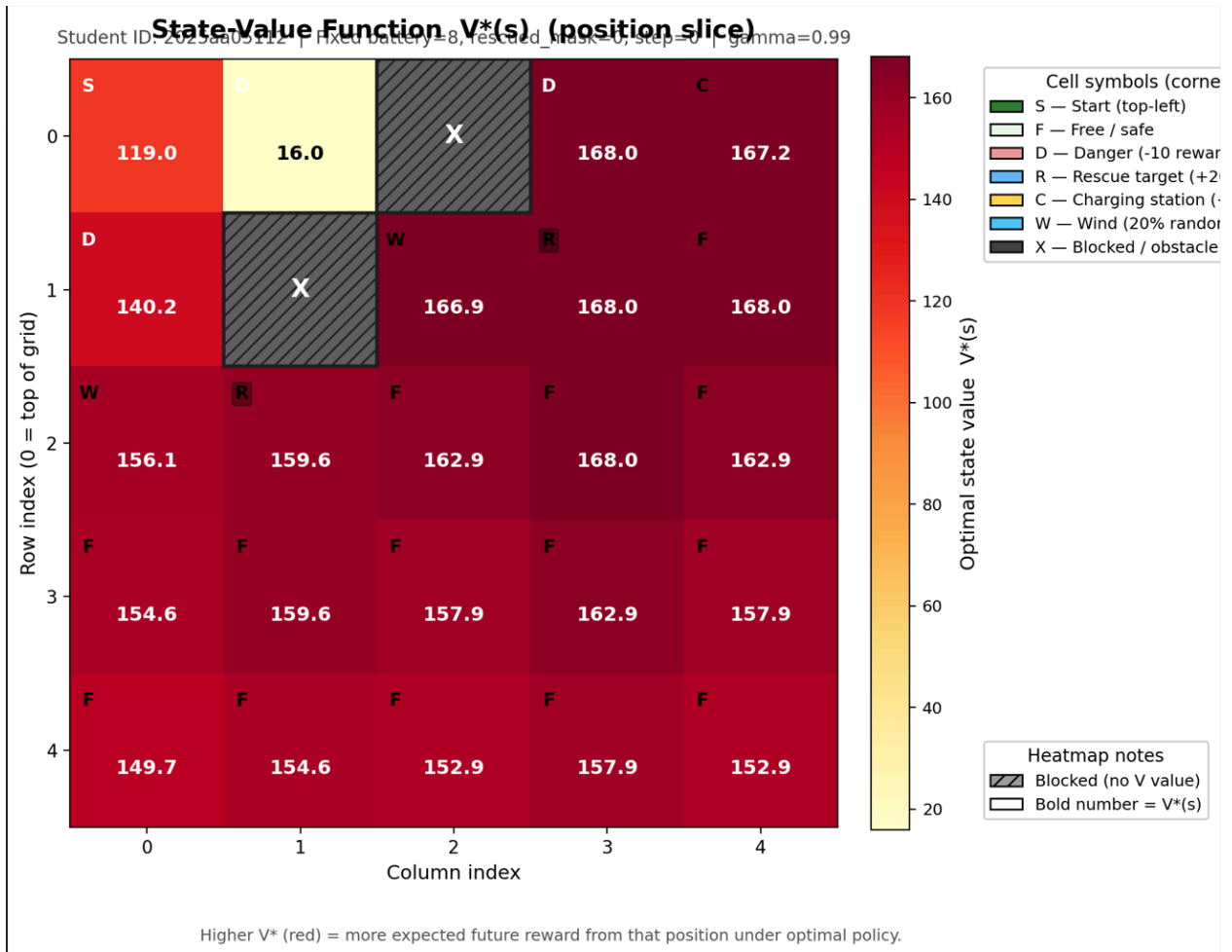
pi*(start) action : Down

Policy plot saved: /home/cloud/SEM2/2025aa05112_policy.png

Value heatmap saved: /home/cloud/SEM2/2025aa05112_value_heatmap.png



Arrows show greedy action per cell. Empty/blocked cells cannot be entered. Wind probability on W cells: 20%.



```
=====
STATE-VALUE ANALYSIS (patterns)
=====
```

Heatmap and policy slices use fixed rescued_mask=0 and battery=8.

Observed patterns:

1. High V^* near rescue targets (R) – the +20 rescue reward propagates backward.
2. Lower V^* near danger (D) because -10 penalises entering those cells unless a rescue path forces a detour.
3. Charging station (C) raises value in its neighbourhood (+5 entry, full refill) when battery is mid-level, encouraging recharge before long detours.
4. Wind cells (W) reduce certainty; optimal policy may prefer safer routes when battery is low because stochastic drift wastes energy.
5. As rescued_mask increases, remaining targets dominate policy arrows toward the nearest uncleared R.

=====
OPTIMAL POLICY ROLLOUT (environment simulation)

=====

Rescue sequence (action -> position, battery, step reward):

```
('start', (0, 0), 10, 0.0)
```

```
('Down', (1, 0), 9, -11)
```

('Down', (2, 0), 8, -1)

```
('Right', (2, 1), 7, 19)
```

```
('Right', (2, 2), 6, -1)
```

```
('Right', (2, 3), 5, -1)
```

```
('Right', (2, 4), 4, -1)
```

('Up' , (1, 4) , 3 , -1)

('Up' , (0 , 4) , 10 , 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

('Up', (0, 4), 10, 4)

[illegible]

```

('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Up', (0, 4), 10, 4)
('Down', (1, 4), 9, -1)
('Left', (1, 3), 8, 19)
Total episode reward: 185.00
Steps taken          : 50
Rescued mask         : 3

```

```

--- Drone Rescue Grid ---

```

```

Position=(1, 3) Battery=8 Rescued_mask=3 Step=50

```

```

S D X D C

```

```

D X W A F

```

```

W F F F F

```

```

F F F F F

```

```

F F F F F

```

```

Legend: S=Start F=Free D=Danger R=Rescue C=Charge W=Wind X=Blocked A=Drone

```

```

---
```

```
=====
DP SCALABILITY DISCUSSION
=====
```

Current explicit state count (approx.): 51612

Curse of dimensionality:

Value Iteration stores $V(s)$ and updates every state. If the grid grows to 10×10 , positions alone multiply by 4 (relative to 5×5). Adding more rescue targets doubles the rescue bitmask for each extra target. Dynamic weather would add another state variable (or hidden mode), multiplying storage and backup cost per sweep.

For 10×10 with 5 rescues, 3 chargers, and weather modes, $|S|$ can exceed millions – each iteration touches all states and all actions, so runtime grows roughly $O(|S| \times |A| \times \text{iterations})$.

Is DP sufficient?

For this 5×5 teaching MDP, DP is appropriate and finds an exact optimal policy. For real autonomous drones with continuous pose, partial observability, moving obstacles, and high-dimensional sensors, tabular DP is infeasible.

How Deep RL helps:

Function approximation (DQN, PPO, actor-critic) generalises across similar states without enumerating the full grid. Experience replay and simulation (digital twins) scale learning to large state spaces.

Real-world relation:

Disaster-response drones must trade off rescue value, energy, and risk under uncertainty – similar to our wind model. Production systems combine learned policies with safety shields and online replanning rather than offline VI on the full physical state space.

```
=====
ASSIGNMENT PART 2 COMPLETE
=====
```